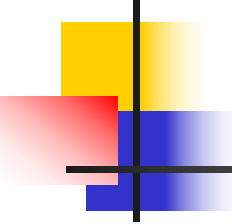


多维数据索引

当数据方体采用关系表方式存储数据时，为了提高查询处理性能，必须采用索引技术。数据仓库环境下，通常采用两种类型的索引：

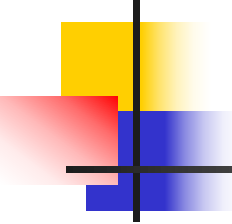
树索引

位图索引



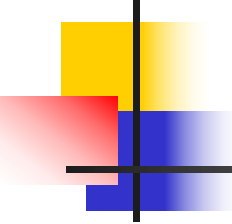
树索引

- B-树索引
- R-树索引
- cube-树索引



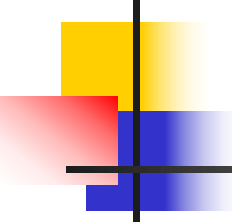
B-树索引

- B-树索引结构自从30多年前被提出以来，久经历史的考验，如今已被应用到各种高级数据库领域，如
 - 数据仓库、多维数据库、空间数据库、时态数据库、多媒体数据库、主存数据库、甚至XML数据库等。



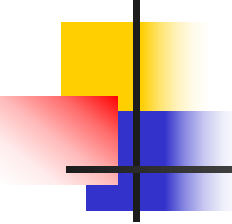
B-树索引

- B-树索引结构自从30多年前被提出以来，久经历史的考验，如今已被应用到各种高级数据库领域，如数据仓库、多维数据库、空间数据库、时态数据库、多媒体数据库、主存数据库、甚至XML数据库等。



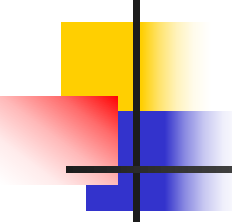
B-树索引

- 虽然针对各种具体应用，人们研究出了很多新的索引方法，但是**B-树索引**在这些应用领域中仍占有一席之地，这主要是基于两个原因：
 - 大多数商业数据库都采用的是**B-树索引**，从兼容性方面考虑，人们仍然偏向于使用**B-树索引**。
 - 基于**B-树**的索引方案最容易实现和理解

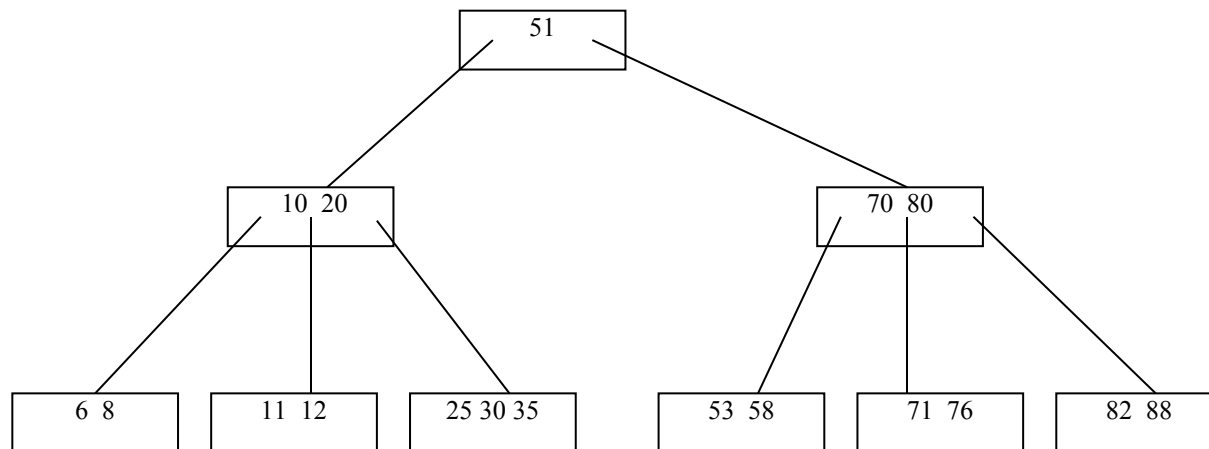


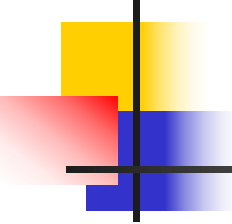
B-树索引

- B-树索引是一棵具有如下特点的多路查找树：
 - 根结点至少有两棵子树（除非根结点同时又是叶子结点）；
 - 每个非根非叶子结点具有 $k-1$ 个关键字和 k 个指向子树的指针，其中 $n/2 \leq k \leq n$ ；
 - 每个叶子结点具有 $k-1$ 个关键字，其中 $n/2 \leq k \leq n$ ；
 - 所有的叶子结点都在同一层上。



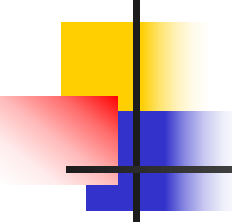
B-树索引





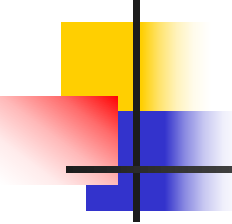
B-树索引

- 由于B-树的每一个结点对应于一个磁盘页，访问一个结点则意味着进行一次I/O操作。因此，希望B-树的结点个数越少越好，
- 为了更好地利用B-树结点的存储能力，Donald Knuth提出了B*-树，它是B-树的一个变种。在B*-树中，除了根结点外的所有结点都要求至少是三分之二饱和，而不是B-树所要求的半饱和。



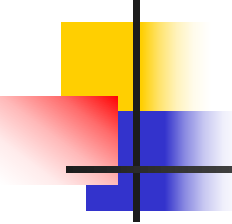
B-树索引

- 尽管B-树结构可以提供快速精确的查找，但是在处理区域查询的时候，需要对B-树进行深度优先遍历，某些内部结点可能需要访问多次。



B⁺-树索引

- 为了减少计算复杂度，B⁺-树被提了出来。
。 B⁺-树和B-树具有如下不同：
 - 所有的关键字都出现在叶子结点，内部结点的关键字用作查找过程中的分界符
 - 所有的叶子结点都被连接在一起，简化了区域查询。



B-树索引

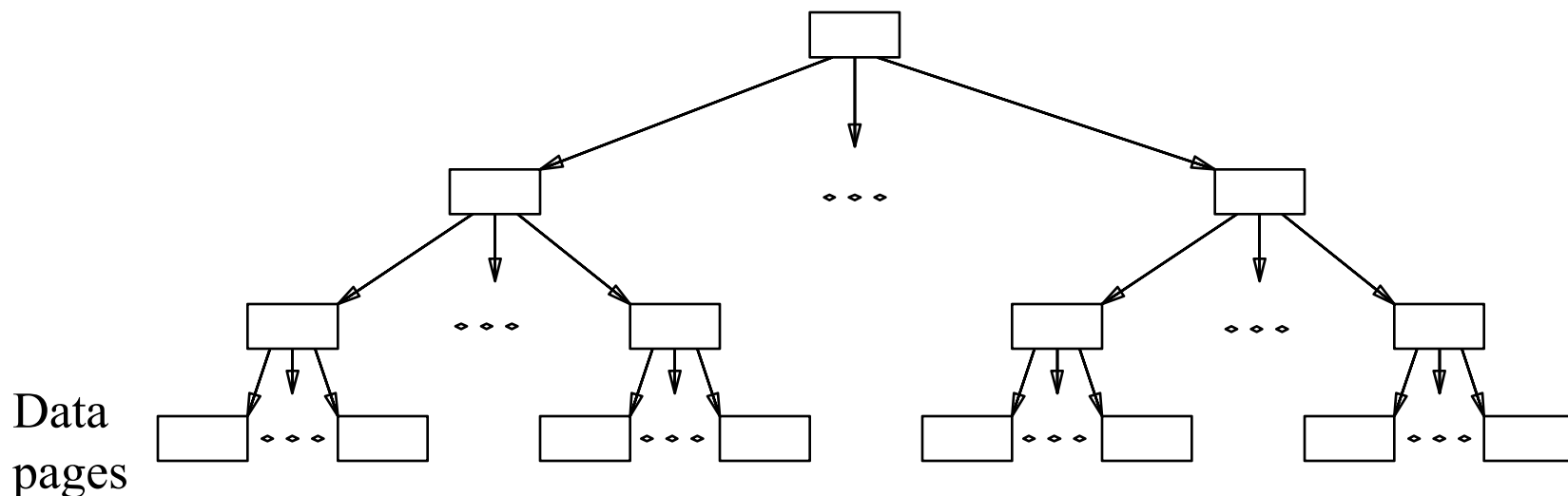
- 但是，要将**B**树索引直接应用于多维数据时，往往需要在多个维上分别建立索引
- 由于各个维的索引彼此之间是独立的,这样将导致关键字必须存储多次，极大地浪费了存储空间。

为此，在**B**树、**B+**树的基础上，人们又提出了专门针对多维数据进行索引的**R**树。

Tree-Structured Indexing

Tree-structured indexing techniques support both *range searches* and *equality searches*

- index file may still be quite large. But we can apply the idea repeatedly!



B⁺ Tree: The Most Widely Used Index

- *Height-balanced.*

- Insert/delete at $\log_F N$ cost (F = fanout, N = # leaf pages);

- *Grow and shrink dynamically.*

- Minimum 50% occupancy (except for root).

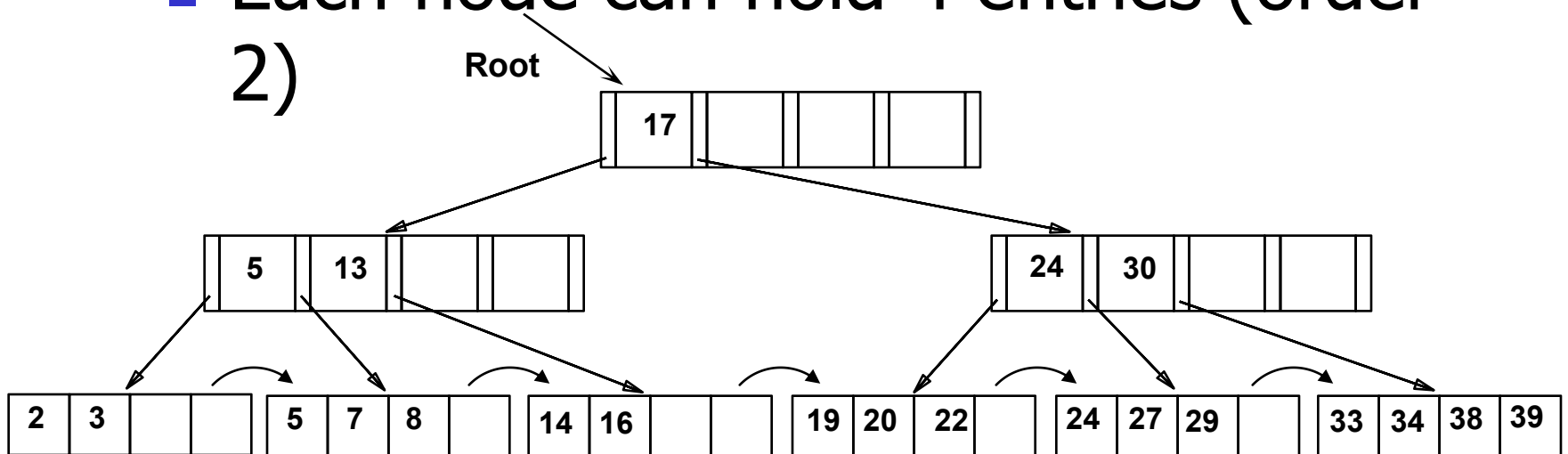
- Each node contains $d \leq m \leq 2d$ entries. The parameter d is called the *order* of the tree.

- ‘next-leaf-pointer’ to chain up the leaf nodes.

- Data entries at leaf are sorted.

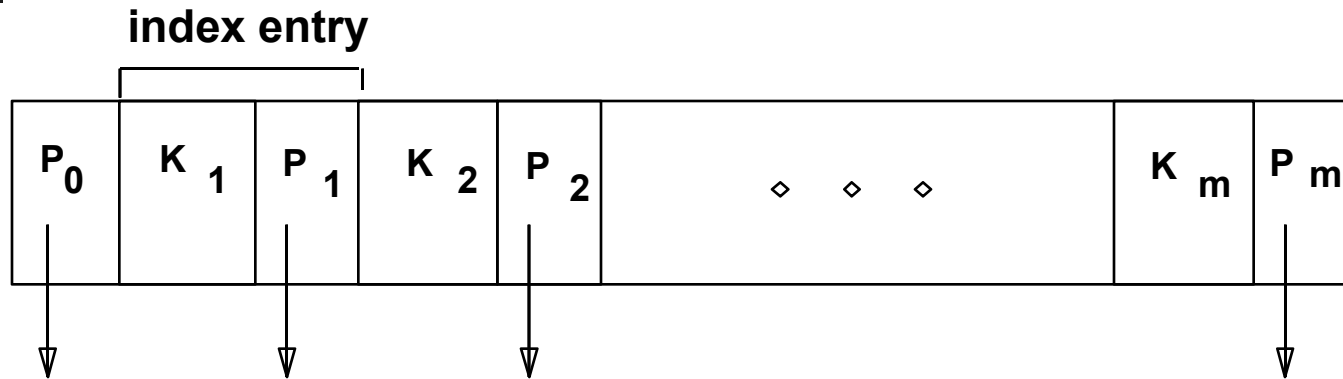
Example B+ Tree

- Each node can hold 4 entries (order = 2)

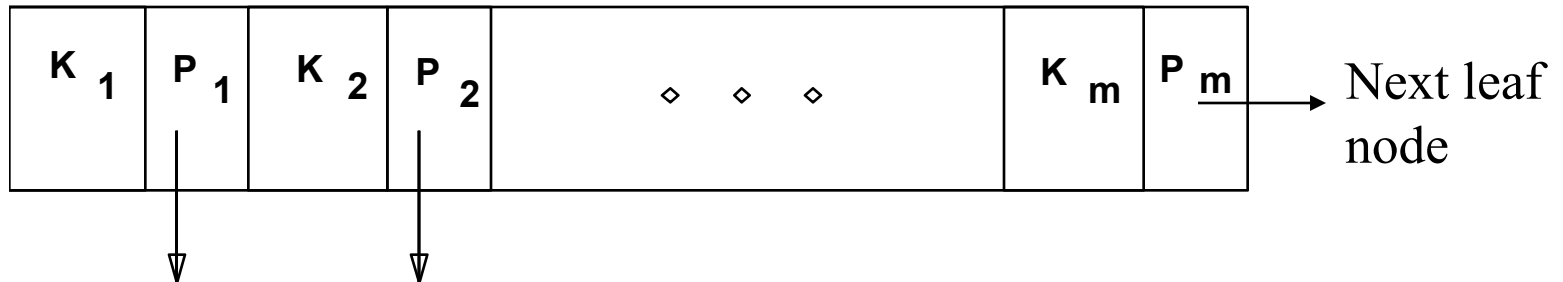


Node structure

- Non-leaf nodes



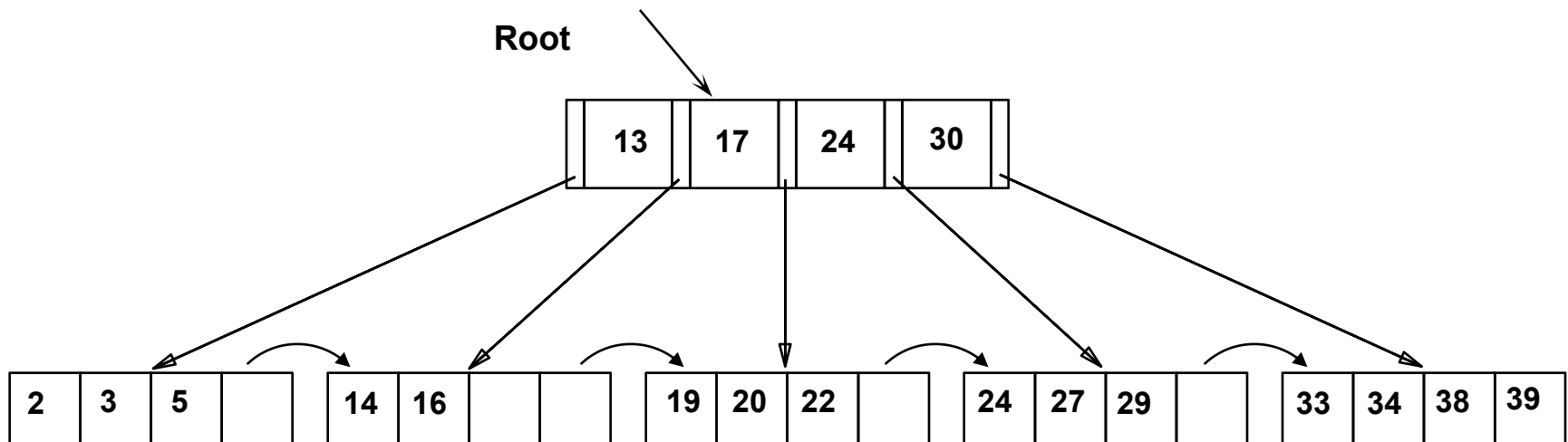
z Leaf nodes



Searching in B+ Tree

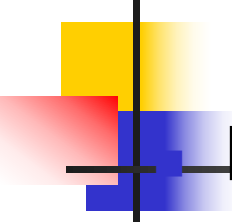
Search begins at root, and key comparisons direct it to a leaf

- Search for 5, 15, all data entries ≥ 24 ...

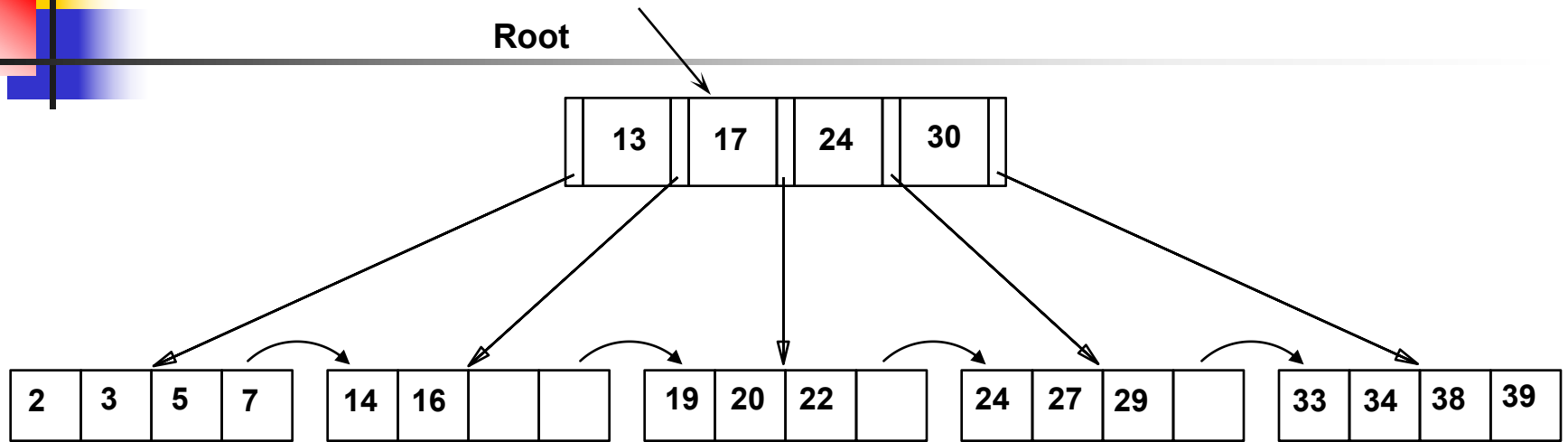


* *Based on the search for 15*, we know it is not in the tree*

Inserting a Data Entry into a B+ Tree

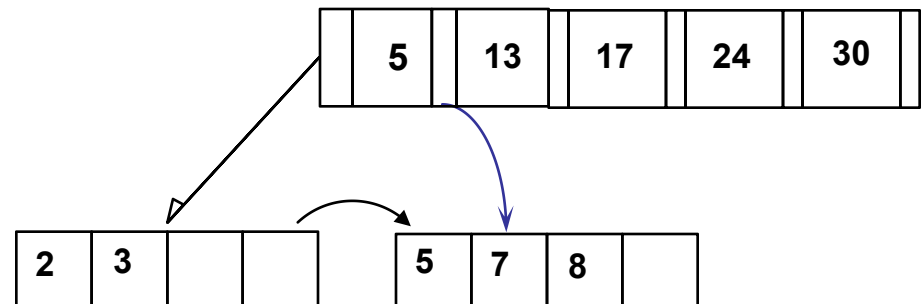
- 
- Find correct leaf L .
 - Put data entry onto L .
 - If L has enough space, *done!*
 - Else, must split L (into L and a new node $L2$)
 - Redistribute entries evenly, copy up middle key.
 - Insert index entry pointing to $L2$ into parent of L .
 - This can happen recursively
 - To split index node, redistribute entries evenly, but push up middle key. (Contrast with leaf splits.)
 - Splits “grow” tree; root split increases height.
 - Tree growth: gets wider or one level taller at top.

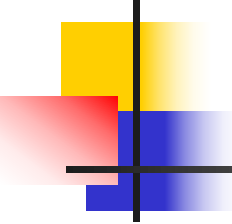
Inserting 7 & 8 into Example B+ Tree



(Note that 5 is copied up and continues to appear in the leaf.)

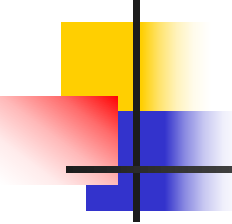
- Observe how minimum occupancy is guaranteed in both leaf and index pg splits.





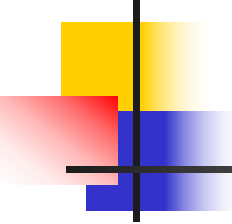
R-树索引

- 和B树、B+树类似，通过提供所要查找数据的一条存取道路（从根结点到某个叶结点的一条路径）为快速查找数据记录提供捷径，并且直接指向包含该数据项的数据页



R-树索引

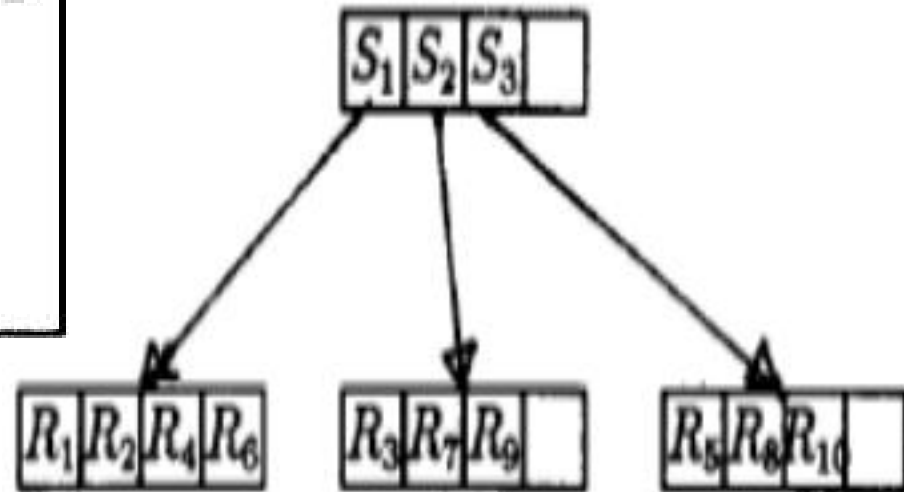
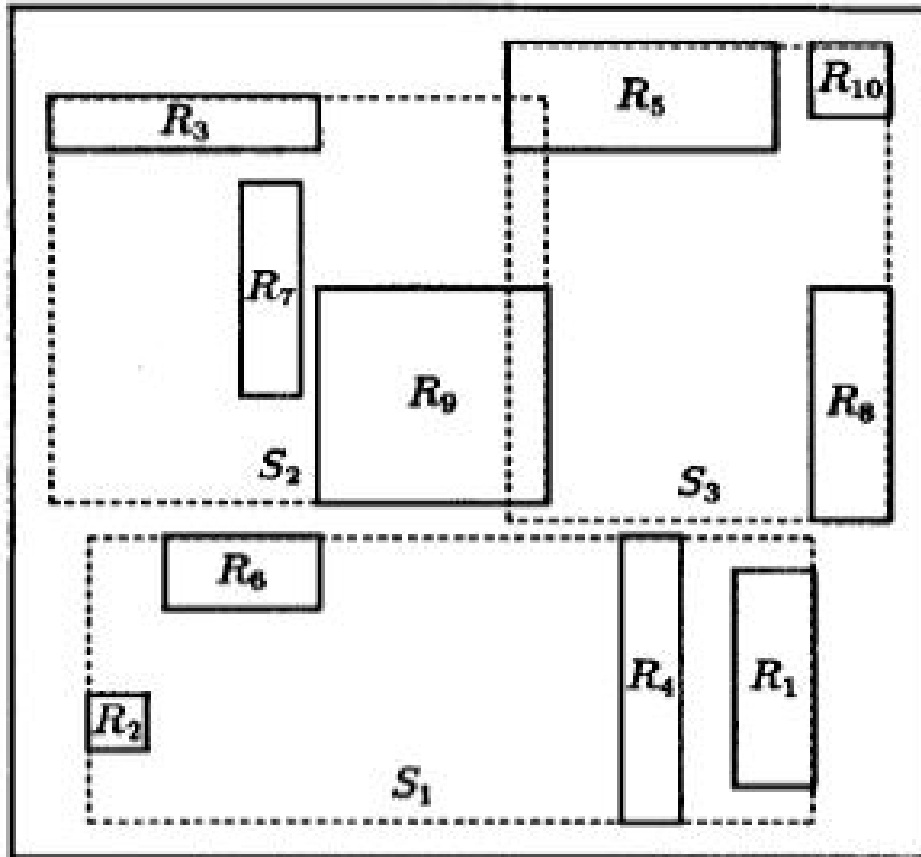
- R树的叶子结点包含指向数据对象的指针，格式为 $(I, \text{对象标识符})$ 。其中
 - $I = (I_1, I_2, \dots, I_n)$ 表示一个 n 维的超矩形，表示数据对象在 n 个维上的边界
 - $I_i = [a, b]$ 表示在某个维上的取值范围

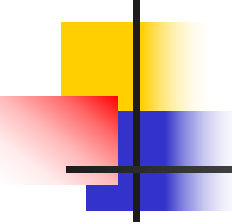


R-树索引

- R树的非叶子结点包含指向下一层子女结点 (**Child**) 的指针, 格式为 (**I**, 子女指针), 其中
 - **I**指向一个可以覆盖下一层所有结点的超矩形
 - 子女指针指向下一层的结点的地址

R-tree (A. Guttman SIGMOD'1984)

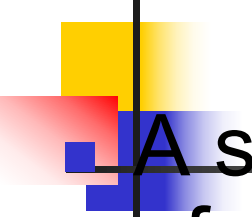




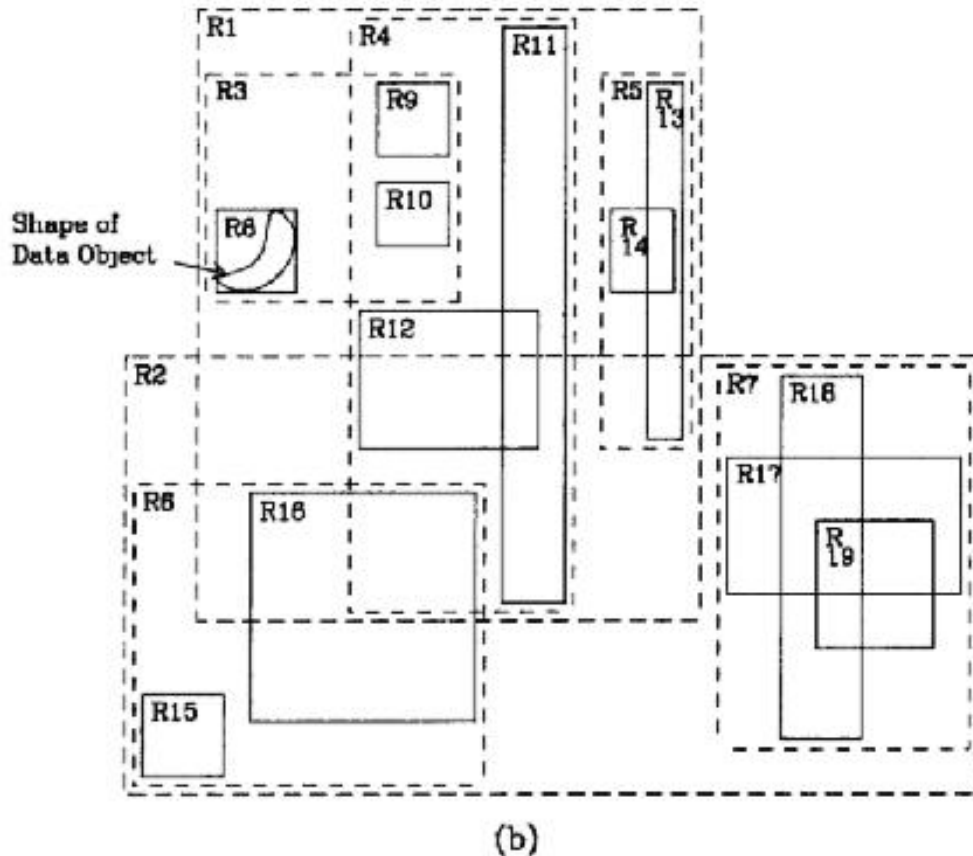
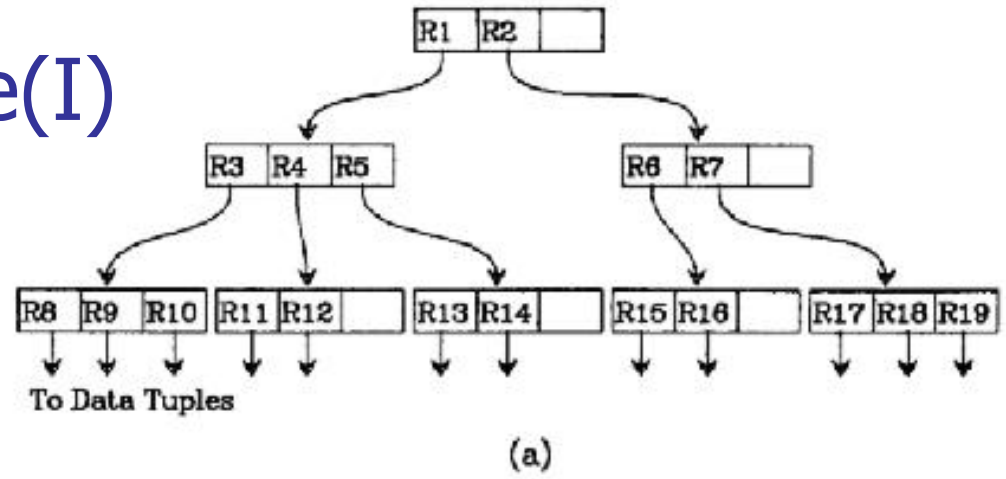
R-tree的特点

- R-tree是B-Tree对多维对象（点和区域）的扩展
- R-tree是一棵平衡树
- 一个多维对象只能被分到一个子空间中去

R-Tree Index Structure

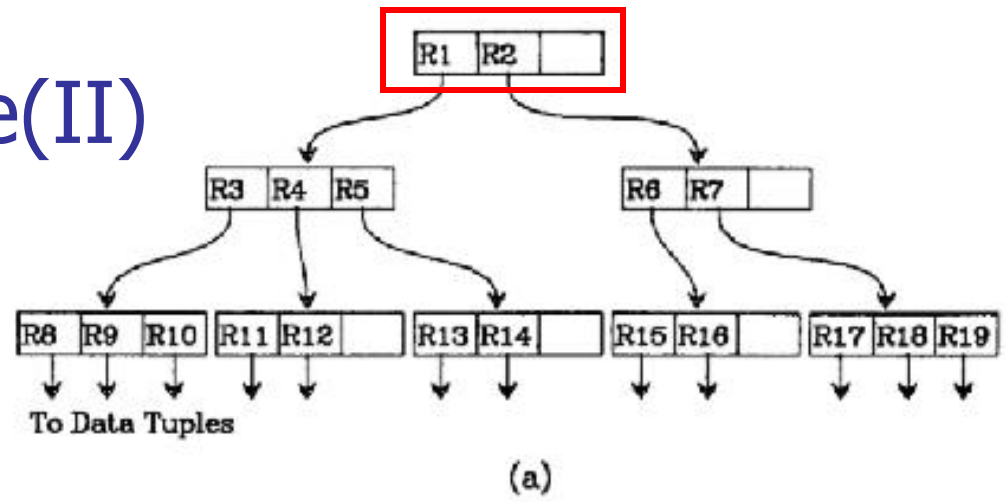
- 
- A spatial database consists of a collection of objects representing spatial objects, where each object has a unique identifier, which can be used to retrieve it.
 - An R-tree is a height-balanced tree similar to a B-tree, with index records in its leaf nodes containing pointers to data objects.
 - Nodes correspond to disk pages if the index is disk-resident.

R-tree: Structure(I)

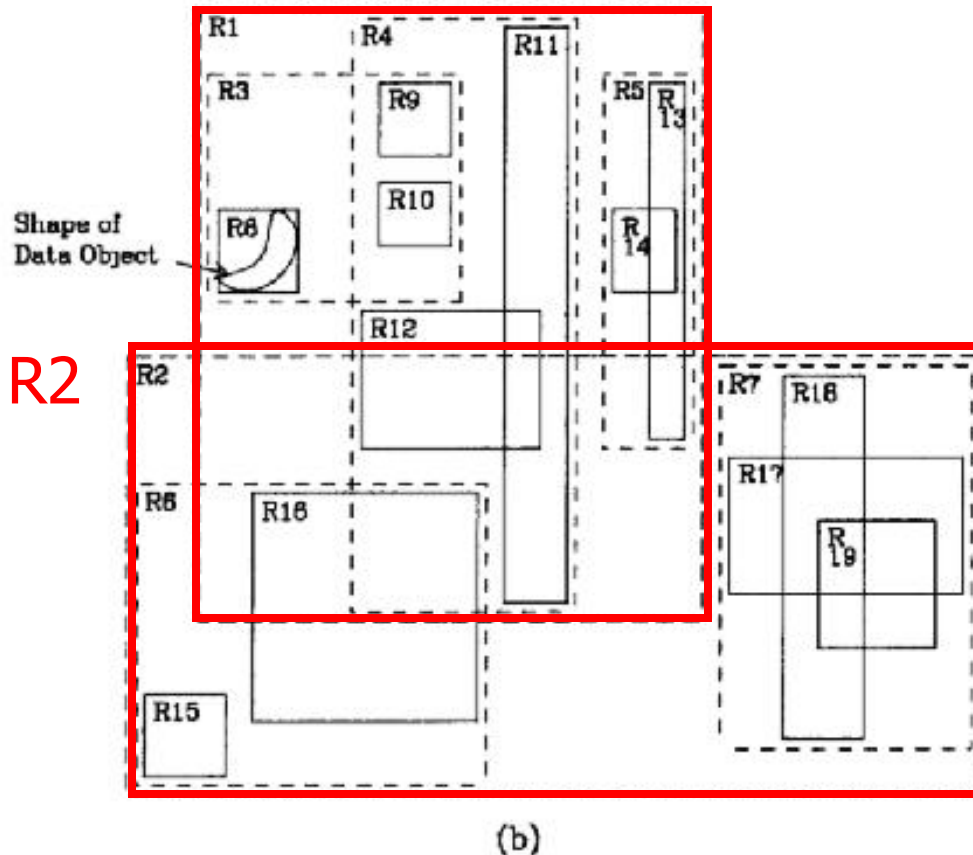


We will transverse through the R-Tree to give some idea of its structure

R-tree:Structure(II)

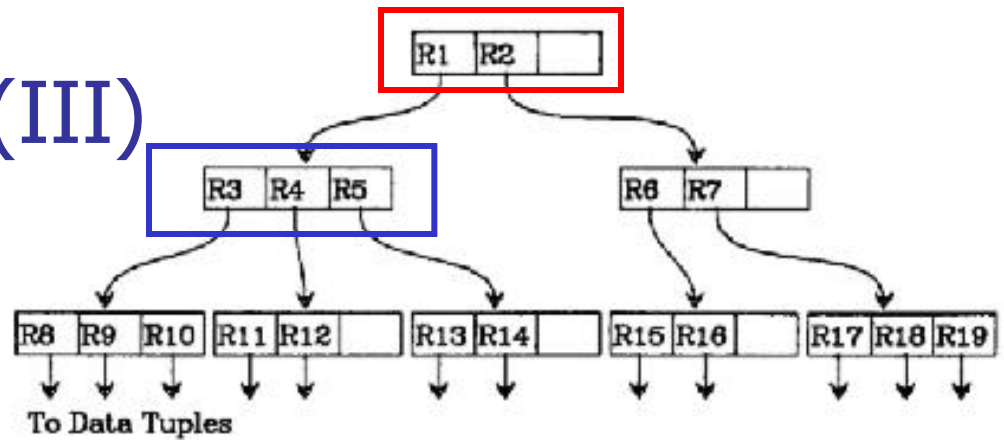


R1

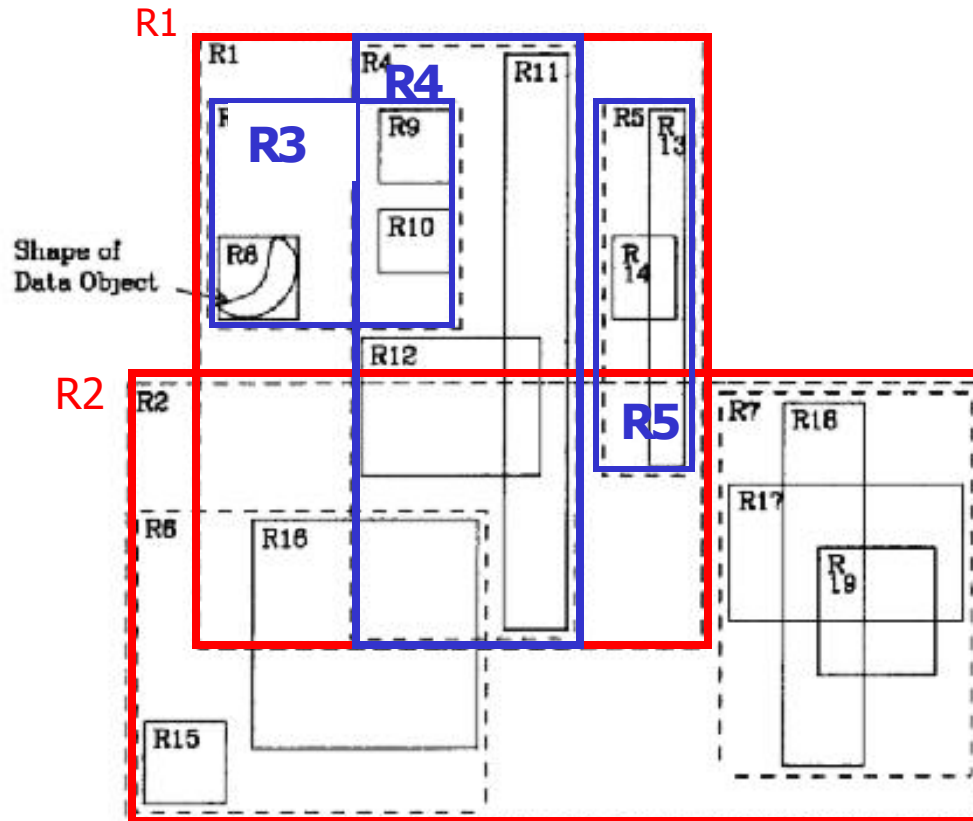


We first move to the first layer of the R-tree which consist of the two nodes **R1** and **R2**.

R-tree:Structure(III)



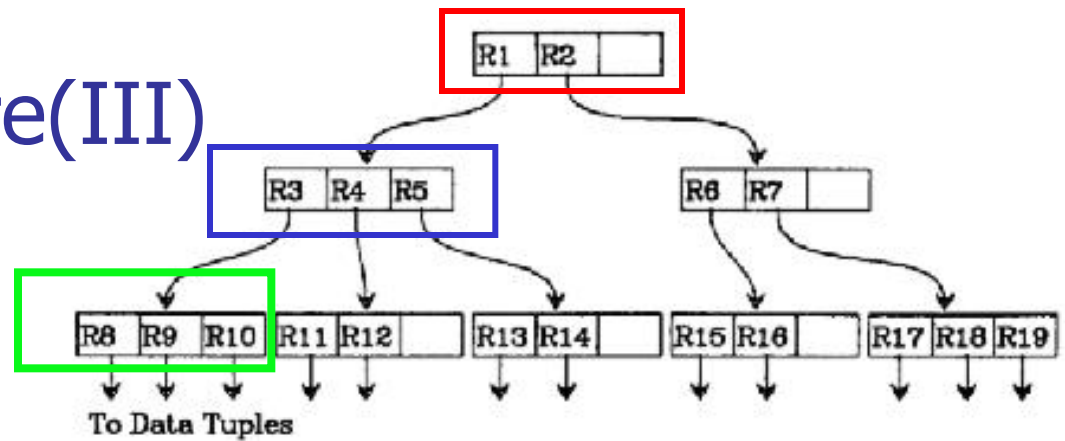
(a)



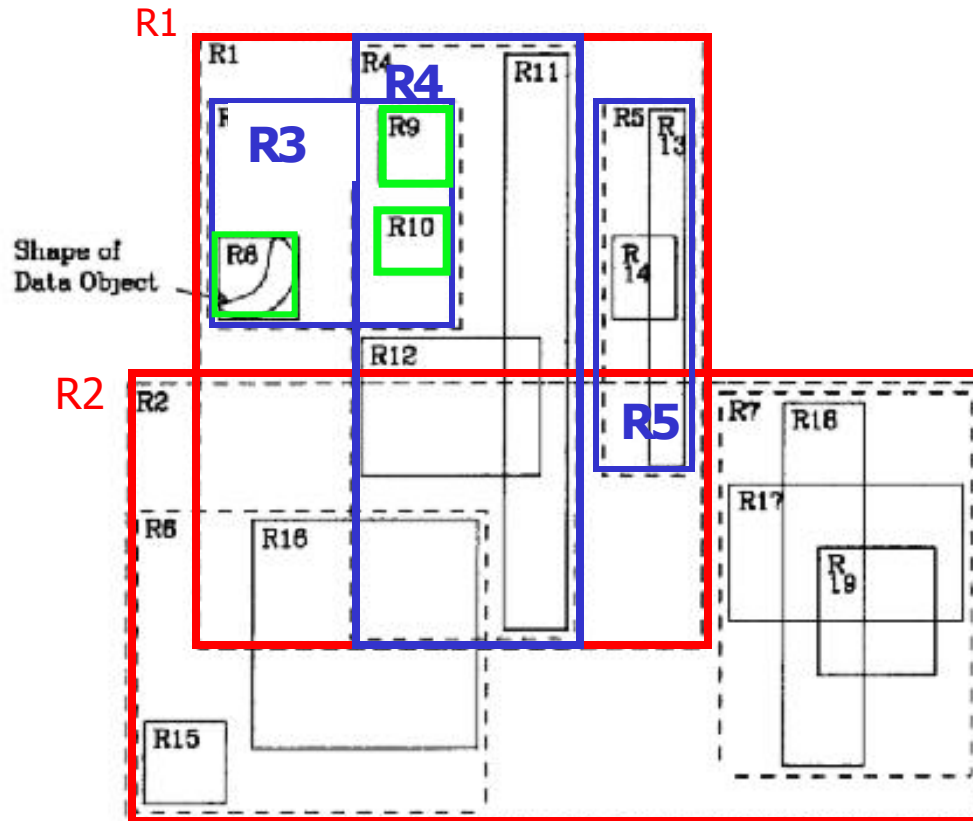
(b)

Then we move to the second layer under node **R1** which consist of node **R3**, **R4** and **R5**

R-tree:Structure(III)



(a)



(b)

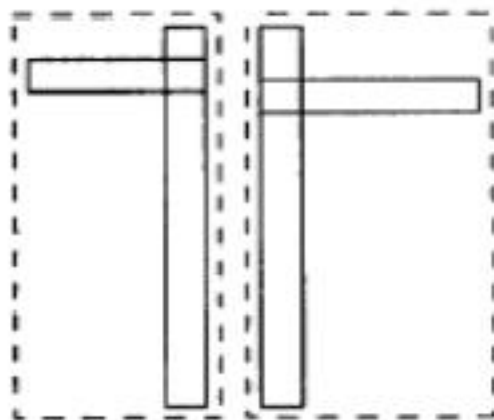
Then we move to the third layer under node **R3** which consist of node **R8**, **R9** and **R10**

Maintenance of R-tree

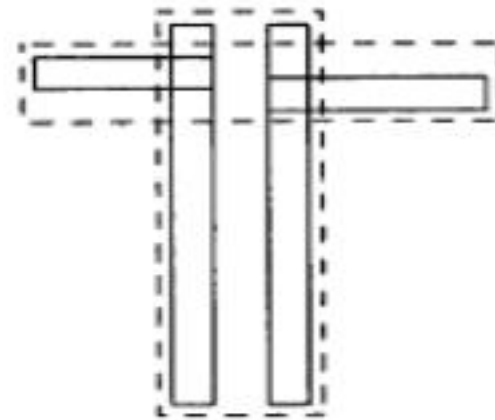
Very much like the maintenance of B+ Tree.

Take average $O(\log n)$ to search, insert and delete.

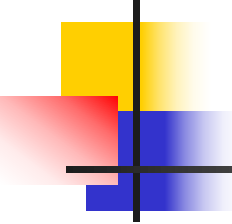
- Concern when splitting nodes: Need to ensure that the bounding box have the smallest area



Bad split

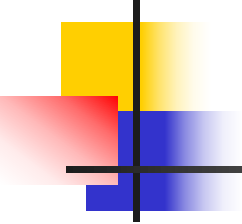


Good split



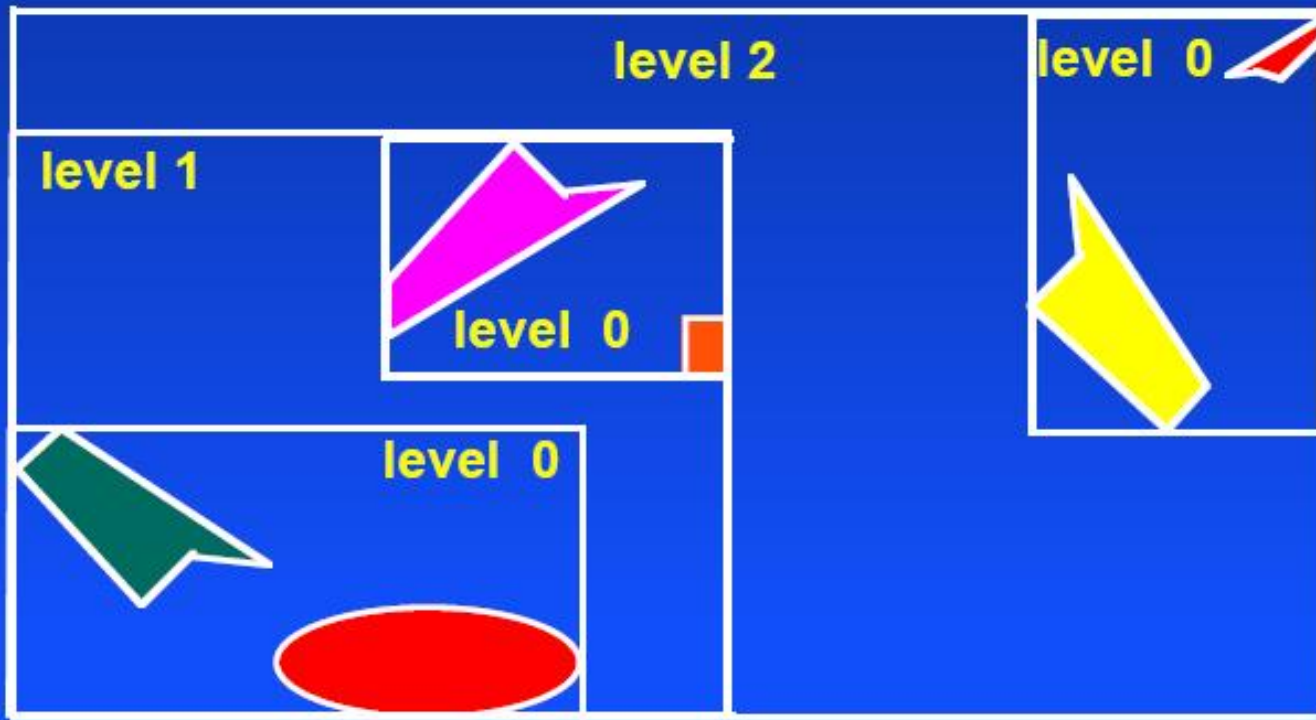
R-tree的典型算法

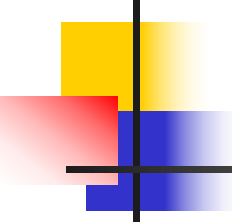
- 查找
- 插入
 - 选择叶子结点
 - 分裂结点 (有多种算法)
 - 调整树
 - 必要时增加树的高度
- 删除
 - 找到包含要删除记录的叶子结点
 - 删除
 - 压缩树
 - 必要时减小树的高度
- 更新
 - 先删除老的记录索引, 在插入新的记录索引

- 
-
- 下载R-tree源代码:
<http://www.superliminal.com/sources/sources.htm>
在该页面上找到**C & C++ Code**, 再找到
Download R-Tree Templated C++ version
 - 注意: 要装MSVC 6 SP5 或 MSVC .Net 2003
 - 介绍R-tree的文章: R-Tree: A dynamic index structure for spatial searching, sigmod 1984

将R-tree用于NN查找

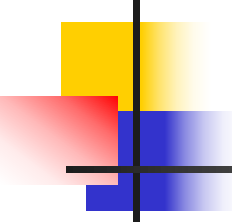
Lemma: Every face of any MBR contains at least one point of an actual spatial object





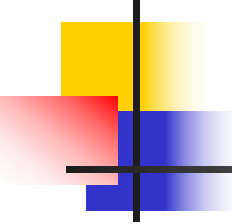
NN查找问题

- 给定空间中的一点 p ，从数据库中找出离 p 最近的点
- 例如：
 - 找出离某家剧院最近的饭店
 - 找出离空中某点最近的星球
 - 找出和某张照片最相似的照片



简单方法:

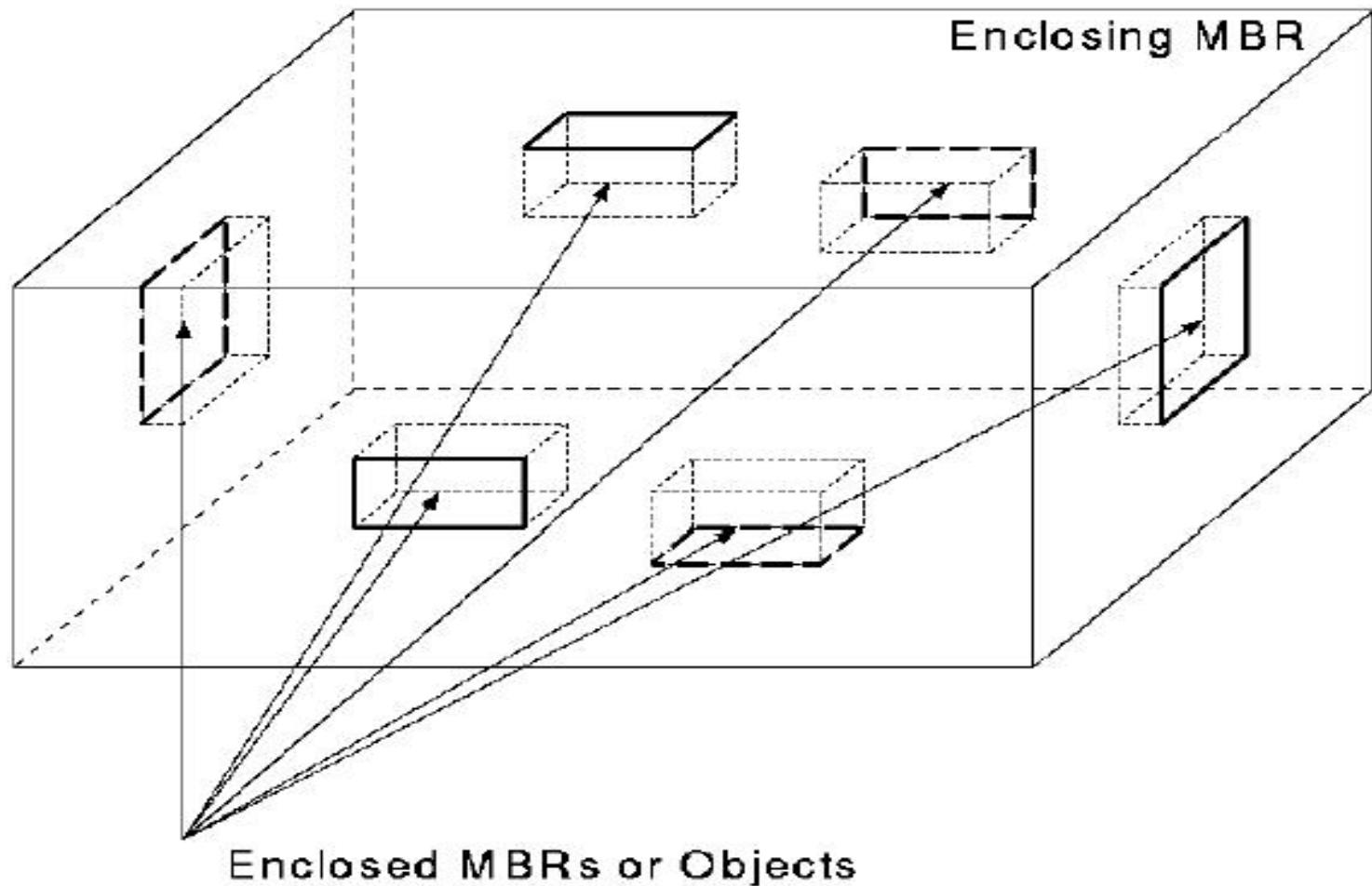
- 逐点比较
 - $\min = \infty$
 - 从集合中取任一点p, 求p和q间的距离
 - If, 小于min, 则更新min
- 缺点: 效率低



改进策略

- 对数据库中的数据建索引，如**R-tree**.
- 采用排除法，排除掉一些根本不可能成为**NN**的点。

The MBR Face Property in 3-d



NN Metrics

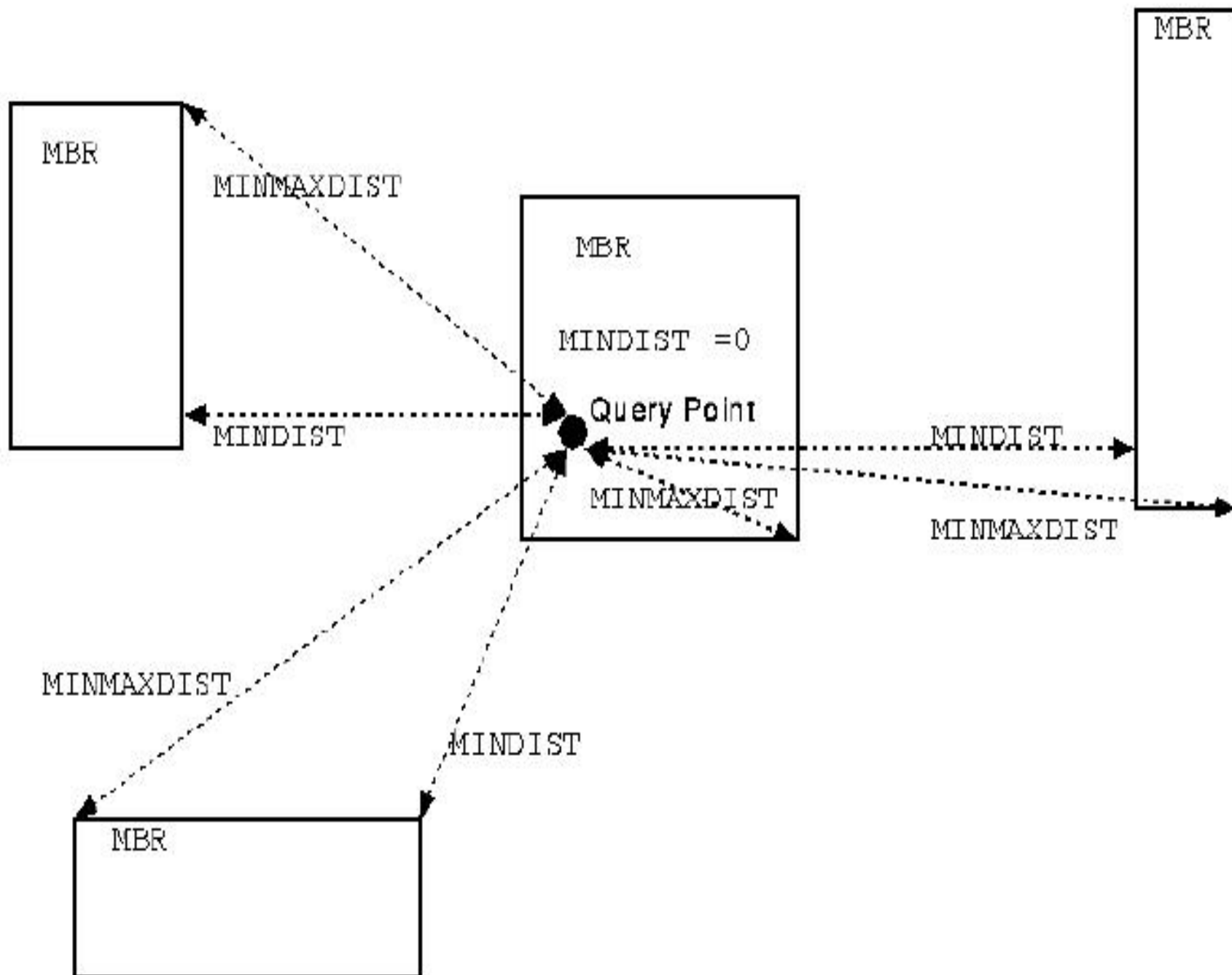
- **MINDIST(P,R):** the shortest distance from P to R

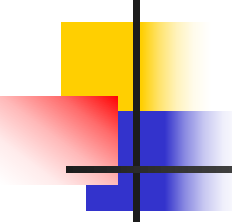
Theorem: Any object O in R has distance from P that is at least as large as MINDIST

- **MINMAXDIST(P,R)** the minimum over all dimensions distance from P to the furthest point of the closest face of the R

Theorem: There exists at least one object within R with distance $\leq$ to MINMAXDIST

$$\text{MINDIST}(P,R) \leq \text{NN}(P) \leq \text{MINMAXDIST}(P,R)$$



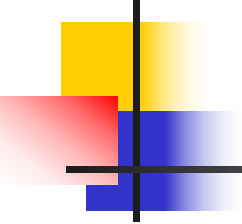


其中: **s**为左下角, **t**为右上角

$$MINDIST(P, R) = \sum_{i=1}^n |p_i - r_i|^2$$

where

$$r_i = \begin{cases} s_i & \text{if } p_i < s_i; \\ t_i & \text{if } p_i > t_i; \\ p_i & \text{otherwise.} \end{cases}$$



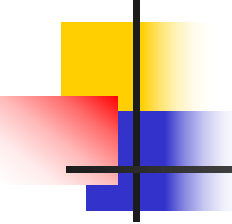
$$MINMAXDIST(P, R) =$$

$$\min_{1 \leq k \leq n} (|p_k - rm_k|^2 + \sum_{\substack{i \neq k \\ 1 \leq i \leq n}} |p_i - rM_i|^2)$$

where:

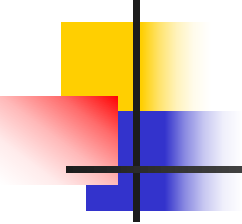
$$rm_k = \begin{cases} s_k & \text{if } p_k \leq \frac{(s_k + t_k)}{2}; \\ t_k & \text{otherwise.} \end{cases} \quad \text{and}$$

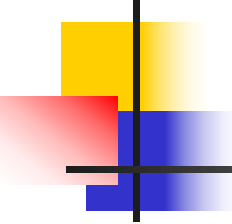
$$rM_i = \begin{cases} s_i & \text{if } p_i \geq \frac{(s_i + t_i)}{2}; \\ t_i & \text{otherwise.} \end{cases}$$



Pruning策略

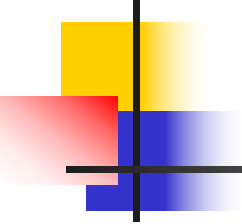
- an MBR R is discarded if there exists an R' s.t.
 - $\text{MINDIST}(P, R) > \text{MINMAXDIST}(P, R')$

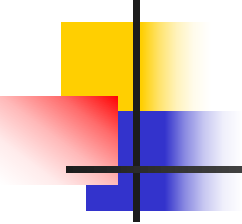
- 
- 实际上，MinDist是下届，MaxMinDist是上届。
 - 查找的过程是拿别人的下届和当前的最小上届进行比较，如果下届大于上届，则删除，否则，继续比较
 - 尽早减小上届。提高算法性能。



算法

- **best first traversal** of the nodes in the R-tree.
- A heap is maintained for storing every MBR

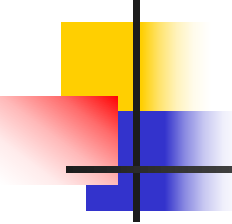
- 
-
- The algorithm first starts from the root MBR of the R-tree and places its children MBRs into the heap
 - Within the heap, the MBRs are ordered based on ascending MinDist

- 
-
- then repeated recursively beginning from the MBR at the top of the heap again by taking its children MBRs and inserting them into the heap.
 - The algorithm maintains a variable **Best** which is initially set to ∞ and are updated later

Input: An R-tree R, query point p

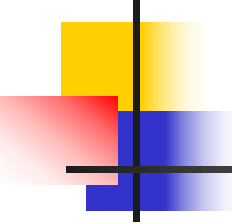
Output: NN of p

1. Best := ∞ ; E = {}
2. Insert the root MBR of R-tree in the heap;
3. WHILE exist MBRs in heap DO
4. Extract an MBR R in heap;
5. IF MinDist(R; p) > Best
6. THEN Exit; //Handle next R on heap
7. ELSE IF MinMaxDist(R; p) < Best
8. THEN Best = MinMaxDist(R; p);
9. IF R is not a leaf MBR THEN
10. Extract all children of R and add to heap, sort the heap;
11. ELSE add all points in R to E
12. Compute nearest dominator q among points in E;
13. Output q;



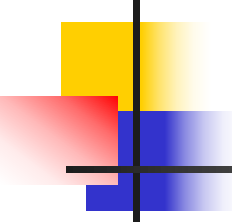
R-树索引

- R树和B树的最大区别在于：
 - 在R树中，兄弟之间的矩形边框可以互相重叠。
 - 如果将R树用于点查询的话，该点可能会同时落到多个子女的矩形边框中。这时需要对所有的这些点继续进行搜索。



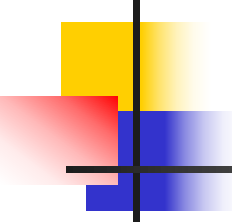
R-树索引

- 为了提高查询的效率，必须尽量减少兄弟之间的这种重叠，为此提出了压缩的R树（**packed Rtree**）的概念。
- 压缩的R树采用空间压缩技术，最大限度地消除了R树中的重叠和死角（没有数据点的空间）现象，从而提高了数据搜索的效率。



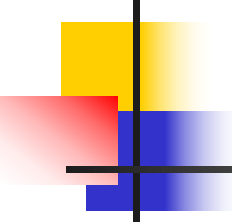
R-树索引

- 通常R树中的每个结点对应一个磁盘页面，因此需要仔细选择参数，使得R树在用于查询处理时只需存取少量的结点。
- R树索引是一种动态的数据结构，查询、插入、删除可以同时进行，不需要周期性地对它进行刷新和重建。



cube-树索引

- 采用**B-树索引**或者**R-树索引**来实体化视图时，受到两方面的限制
 - 索引和数据是分离的，关键字必须存储多次，浪费了存储空间
 - 关系存储中，数据没有聚簇存放，这样在对数据进行维护时就无法进行快速的合并操作

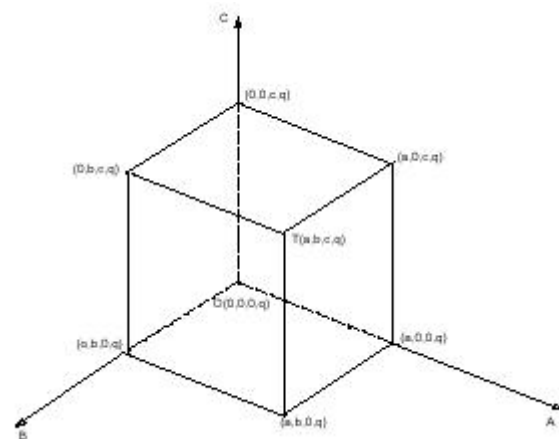


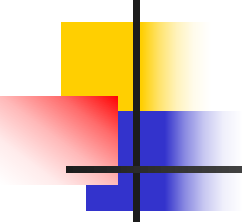
cube-树索引

- **cube-树**将数据和索引的存储融为一体，获得了较高的查询性能。
- 与传统的**OLAP**实体化视图的存储组织结构相比，**cube-树**最好可以使存储空间缩减**2**倍，查询性能提高**10**倍，批量更新速度提高**100**倍。

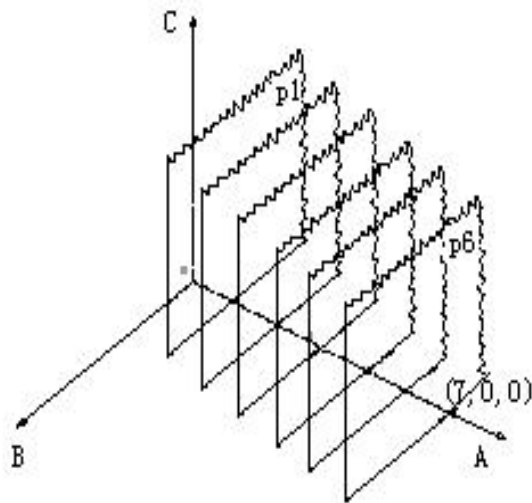
cube-树索引

- 假设事实表 $R(A,B,C,Q)$ 由三个维属性 A 、 B 、 C 和一个度量属性 Q 组成。
- 将三个维属性分别映射到多维空间的 X 、 Y 、 Z 坐标轴上，则事实表 R 中的每一个元组 $T(a,b,c,q)$ 都可以映射为多维空间 $A \times B \times C$ 的一个点。
- 该点的内容为 q ，在 X 、 Y 、 Z 轴上的坐标分别为 a 、 b 、 c



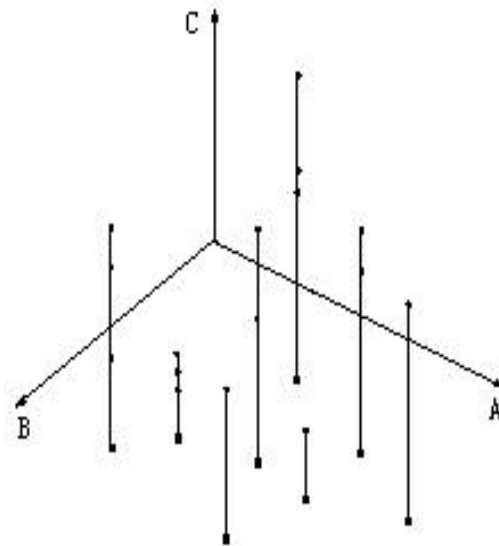
- 
-
- 将多维空间中的数据点投影到 $A \times B \times C$ 的任何一个子空间上，就可以得到不同的聚集值。

cube-树索引



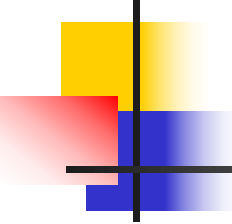
(a)

图6.4 (a)在GroupBy(A)上的投影;



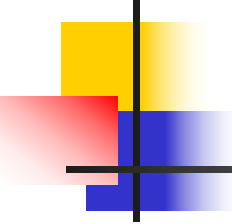
(b)

(b) 在GroupBy(A, B)上的投影[RKR97]



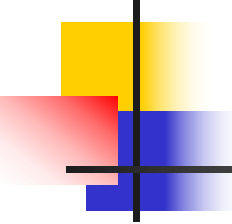
cube-树索引

- 这种方法将事实表及其所有的聚集统一地以可视化的方式表示出来，被称为扩展的数据方体模型(Extended Datacube Model, 简称EDM)。



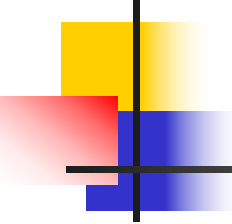
cube-树索引

- 任何关系的或多维的存储结构都可以用来实现EDM。例如
 - 可以采用不带任何索引的传统的关系表来实现
 - 也可以采用R-树来实现
 - 或采用关系表和R-树以及B-树的组合来实现



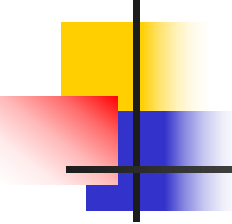
cube-树索引

- 由于这种索引结构是树形的，将这种EDM模型称为cube-树。
- 它可以用来存储ROLAP视图。任意集合的ROLAP视图都可以映射为一组cube-树



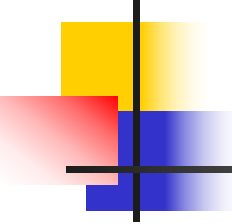
Bitmap索引

- 树索引在叶子结点针对每个关键字存储一个**RID**（记录号）列表。
 - 对于**基数较大**的属性，如订单号、顾客标识等，每个关键字所对应的**RID列表通常比较短**，查询时需要存取较少的数据，因此效率较高。



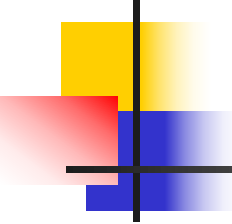
Bitmap索引

- 将树索引应用于**基数较小**的属性如性别、民族时，每个关键字所对应的**RID列表很长**，查询时要存取大量的数据页，几乎相当于顺序扫描，索引所带来的好处微乎其微。



Bitmap索引

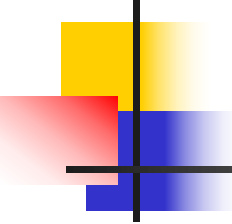
- 对于**基数较小**的维属性，更适合采用另一种称之为**bitmap**的索引。
- 全称是基于**Bitmap**的**B+树索引**（**Bitmap-based B+ -Tree**），简称**bitmap索引**。
- 本质上仍是一种**B+树索引**，不同的是在叶子结点不再存储**RID列表**，而是存储一个**bitmap**（位序列）



简单的Bitmap索引

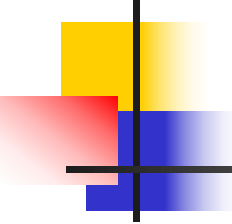
- 首先看下面这张反映顾客信息的表：顾客（顾客号，姓名，性别，重要级别）

顾客号	姓名	性别	重要级别
11	王平	男	3
12	李强	男	5
13	张岚	女	5
14	刘行	男	4



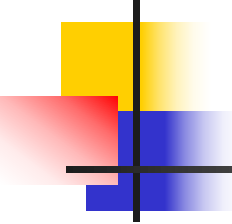
简单的Bitmap索引

- 重要级别，取值范围：1到5，它表示该顾客的重要程度。
- 性别，顾客的性别，只有两个值，男或者女。
- 像这样的取值范围较小的列我们称为稀疏列。



简单的Bitmap索引

- 对于稀疏列可以通过创建特殊的索引来加快在该列上的查询速度。
- 基本思想是对**每一个列值，给它创建一个相应的由0和1组成的序列**，例如
 - 可以用**10**代表男，用**01**代表女
 - 同理，**10000**代表重要级别为1，**00001**代表重要级别为5。



简单的Bitmap索引

顾客号	姓名	性别	重要级别
11	王平	男	3
12	李强	男	5
13	张岚	女	5
14	刘行	男	4

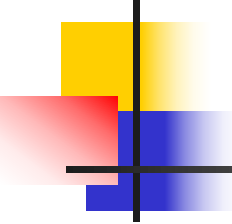
(a)

男	女
1	0
1	0
0	1
1	0

(b)

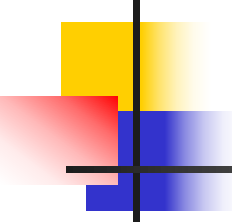
1	2	3	4	5
0	0	1	0	0
0	0	0	0	1
0	0	0	0	1
0	0	0	1	0

(c)



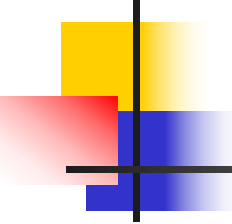
简单的Bitmap索引

- 性别列的取值构成两个位向量，其中一个表示“男”，另一个表示“女”，把这两个位向量一起称作顾客信息表性别列的Bitmap索引
- 重要级别列的5个位向量构成了顾客信息表重要级别列的Bitmap索引



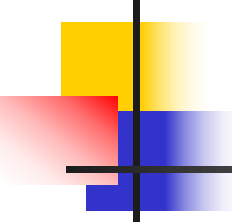
简单的Bitmap索引

- 查询：有多少男性顾客的重要级别为5？
 - “男”：所在列Bitmap为“1101”
 - “重要级别5”：所在列Bitmap为“0110”
 - 将这两列的Bitmap做一个按位“与”操作，得到“0100”。
 - 因为该结果的Bitmap中只有一个1存在，由此可知只有一位男性顾客的重要级别为5。而且根据结果Bitmap中1所出现的位置，可以知道，该男性顾客的名字叫“李强”



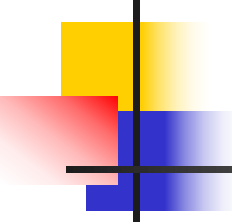
Bitmap索引具有两个优点

- 较紧凑，节约空间：位图索引的大小与列的不同值的个数成正比。
 - 当列的不同值的个数很小（低基数）时，可以节约空间。例如性别这样的属性，位图索引只有2列。
 - 对于高基数的列，**Bitmap**索引并不有效，其解决办法一是进行压缩，二是把它转化成树索引。



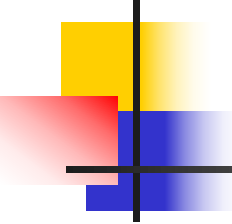
Bitmap索引具有两个优点

- 可以使用有效的位操作来快速回答查询
 - 使用**Bitmap**索引只需扫描一次位图就可对所有的行通过位操作（与/或）进行定位，比通常处理此类查询时进行的全表扫描要快得多



编码的Bitmap索引

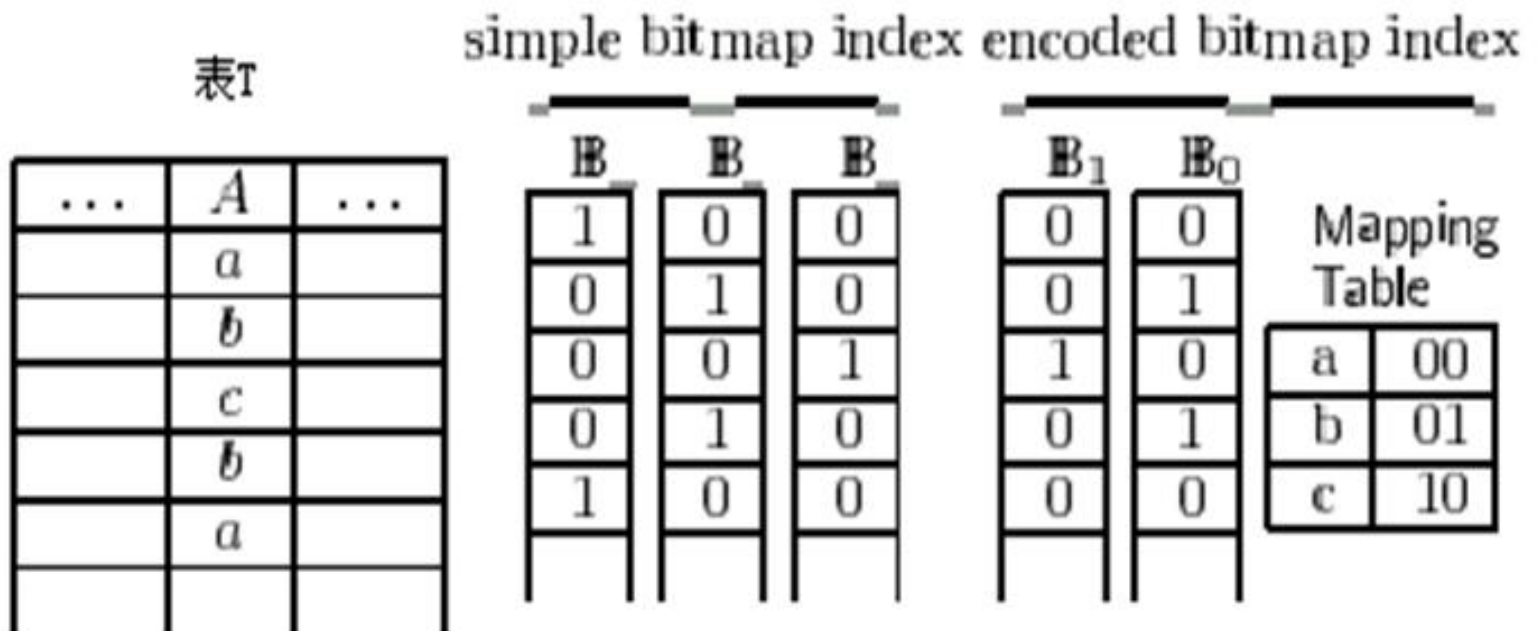
- 对于高基数的列，简单的Bitmap索引不够有效，为此，提出了一种新的索引技术----编码的Bitmap索引

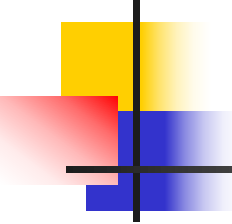


编码的Bitmap索引

- 假定某个表T的属性A的值域为{a, b, c}
- 和简单Bitmap索引不同的是，编码Bitmap索引采用2位 (bit) 而不是3位来对属性A的值域进行编码，值a用编码00来代表，值b用01来代表，值c用10来代表。

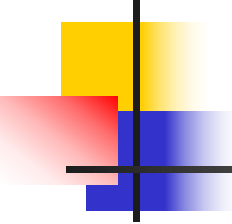
编码的Bitmap索引





编码的Bitmap索引

- 采用编码技术可以大大减小Bitmap索引位向量的个数
 - 如果某个属性列的基数为C的话，简单Bitmap索引所需要的位向量的个数为C个，
 - 编码Bitmap索引所需要的位向量的个数为 $\log_2 C$ ，当然，编码Bitmap索引还需要加上一个编码映射表

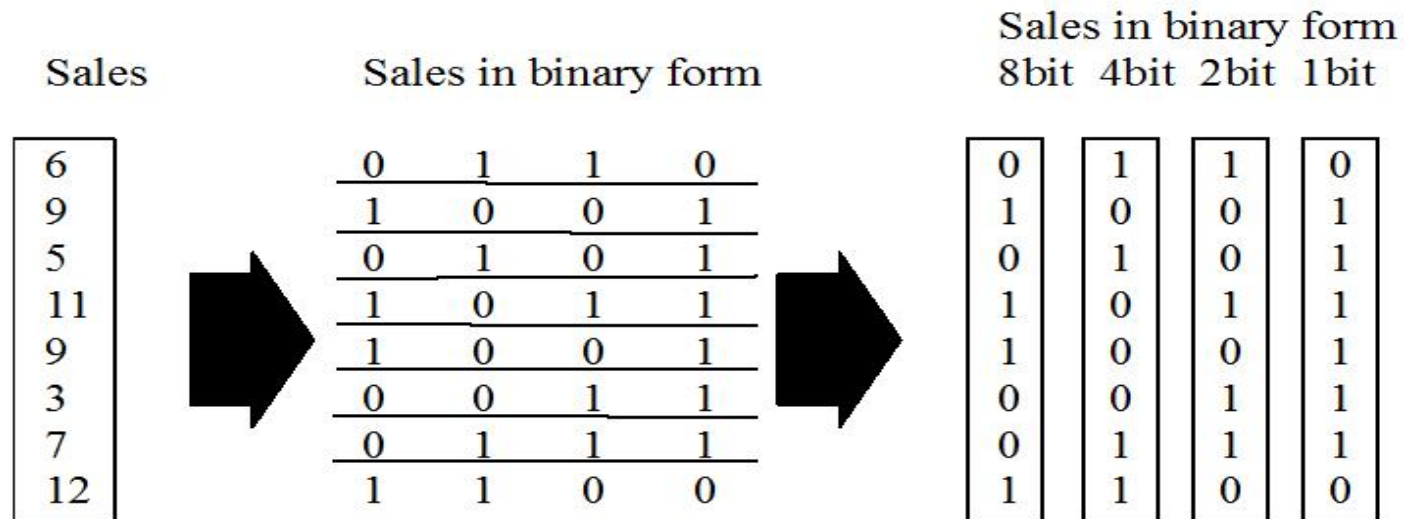


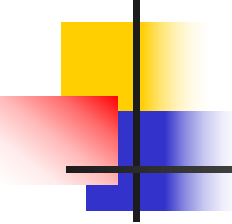
Projection索引

- 思想非常简单
- 将某个表的某一系列以相同的元组顺序冗余存储
- 这样做的好处是查询时如果不需要的话可以不读取其它列的值

Bit-Sliced索引

- 将Projection 索引按照二进制形式存储再进行按位分割之后所形成的每一列就是一个 Bit-Sliced索引





Bit-Sliced索引

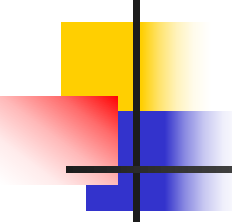
- Bit-Sliced索引所需的存储空间很少，一般只是原始数据空间的10%-20%
- 例如，上例Bit-Sliced索引用的空间是

$$(\text{记录数}) * (\text{索引所需的位数}) = 8 * 4 = 32$$

原始数据空间是：

$$(\text{记录数}) * (\text{存储一个整数所需字节数}) * (\text{存储一个字节所需位数}) = 8 * 4 * 8 = 256$$

索引所占空间是原始数据空间的12.5%



Bit-Sliced索引

- 当需要将该列的所有数据做聚集计算（比如 sum）时，根据所建的Bit-Sliced索引，只需要统计一下每列的1的个数并将其转换为十进制数即可
- 例如，图6.4中，第4列有4个1，第3列有4个1，第2列有4个1，第1列有6个1。由此可得该列的聚集和是
$$4*2^3+4*2^2+4*2^1+6*2^0=62$$
- 由此可见，Bit-Sliced索引非常适合于做聚集查询